



Hybrid Energy Management System

for Electric Vehicles

Project Documentation · Technical Report

MATLAB

Simulink

Simscape Electrical

Stateflow

■ HEMS_in_EV_Model.slx

A complete rule-based Energy Management Strategy implemented in MATLAB/Simulink,
using Stateflow for SOC-based mode switching (Battery → Hybrid → Engine)
and Simscape Electrical for physical power-network simulation.
Features smooth power transitions via first-order blending ($k = 0.2$).

Table of Contents

1. Abstract	4
2. Introduction	5
3. Literature Review	5
4. Problem Statement	7
5. Objectives	7
6. System Architecture	8
7. Components Used	9
8. MATLAB / Simulink / Simscape Implementation	10
8.1 Gain1.....	10
8.2 Gain.....	10
8.3 Saturation.....	11
8.4 Low-Pass Filter (Discrete or Continuous).....	11
8.5 Repeating Sequence (Load Profile).....	13
8.6 Integrator (SOC State).....	13
8.7 Capacity (Constant).....	15
8.8 Resistor (Simscape Electrical).....	15
8.9 Solver Configuration.....	16
9. Energy Management Strategy (EMS)	17
9.1 State: Battery (initial state).....	18
9.2 State: Hybrid.....	18
9.3 State: Engine.....	19
9.4 Summary of Transition Logic	19
10. Working Principle	20
11. Simulation Setup	21
12. Results and Discussion	22
Observed Trends	22
13. Advantages	24
14. Limitations	24
15. Future Scope	25

16. Applications	25
17. Conclusion	25
18. References	27

This report is based entirely on the supplied Simulink/Simscape model (HEMS_in_EV_Stateflow_Upgraded.slx), the block-parameter screenshots, and the Scope1 simulation output. Author identity fields on the cover page are left blank for the student to complete; a small number of model-configuration details that were not visible in the supplied screenshots (e.g., solver type/step size, the unit of the Capacity parameter, and the signal routed to Scope2) are flagged inline where relevant and should be confirmed against the live model before submission.

1. Abstract

This project presents the design and simulation of a Hybrid Energy Management System (HEMS) for an electric vehicle, developed in MATLAB/Simulink using Simscape Electrical for the power network and Stateflow for the control logic. The energy management strategy is rule-based: a three-state Stateflow chart (Battery, Hybrid, Engine) decides how the instantaneous load power is shared between a battery source and an engine source, with mode transitions governed by the battery's State of Charge (SOC) and the magnitude of the load demand. Within the Hybrid and Engine states, a first-order smoothing factor ($k = 0.2$) blends the battery power command toward a SOC-proportional target, avoiding abrupt step changes in power when the controller switches modes. The electrical network — battery and engine modeled as controlled current sources, connected through a resistor and electrical reference, with voltage and current sensors feeding back into a power-computation and SOC-estimation chain — is implemented entirely using Simscape Electrical components bridged to the Simulink control logic via PS-Simulink and Simulink-PS converter blocks. The model was simulated with a repeating five-point load profile (1000-2200, cycling every 20 s) starting from an initial SOC of 80%. The resulting Scope1 traces show the expected qualitative behaviour of a SOC-based rule strategy: battery power dominates initially and decreases steadily as SOC depletes, while engine power rises from near zero and stabilizes as the controller shifts from the Battery state through Hybrid into the Engine state over the simulation horizon. The work demonstrates a complete, modular Simulink/Simscape/Stateflow implementation of a rule-based HEMS, separating the physical electrical network from the decision logic so that the energy management strategy can be modified independently of the underlying power model.

2. Introduction

The transportation sector is a major contributor to fossil-fuel consumption and greenhouse-gas emissions, which has driven sustained interest in hybrid and electric vehicle technology as a way to reduce dependence on the internal combustion engine while retaining the range and refuelling flexibility that a purely electric powertrain cannot yet match everywhere. A hybrid electric vehicle (HEV) combines two or more power sources — typically a battery-fed electric drive and an internal combustion engine — and the manner in which power is split between these sources at every instant has a direct bearing on fuel economy, battery life, and drivability [1], [2].

This division-of-power problem is addressed by what the literature calls an Energy Management Strategy (EMS). EMS approaches are broadly classified into rule-based strategies, which use predefined logical conditions or look-up tables derived from engineering experience, and optimization-based strategies, which compute a power split that minimizes a cost function (fuel consumption, emissions, or battery degradation) either offline or in real time [2], [3]. Rule-based strategies remain widely used in practice because they are simple to design, easy to implement on embedded controllers, and do not require a drive cycle to be known in advance, even though they generally do not guarantee a globally optimal power split [3], [4].

This project implements a rule-based EMS for a simplified battery-engine HEV power network. The objective is not to propose a new EMS algorithm, but to demonstrate, end-to-end in MATLAB/Simulink, how such a strategy is structured, parameterized, and verified using a Stateflow chart for the decision logic and Simscape Electrical for the underlying power network — addressing the specific sub-problem of deciding, at every instant, how much of the load power should be drawn from the battery versus the engine, based on SOC.

3. Literature Review

Wirasingha and Emadi [1] classify control strategies for plug-in hybrid electric vehicles and note that most practical controllers fall into rule-based, fuzzy-rule-based, or optimization-based categories, with rule-based controllers remaining the most common choice for production vehicles because of their low computational overhead and deterministic behaviour.

Xue et al. [2] provide a multi-level classification of HEV powertrains and the corresponding EMS/control algorithms, and observe that for any given powertrain configuration, the EMS choice is ultimately a trade-off between implementation simplicity (favouring rule-based methods) and achievable efficiency (favouring optimization-based methods such as dynamic programming or model predictive control).

Sabri, Danapalasingam, and Rahmat [3] review HEV architectures and EMS techniques and describe rule-based control as relying on if-then logic or look-up tables that use thresholds on quantities such as battery SOC and power demand to decide the operating mode of the vehicle — this is structurally the same approach used in the present project's three-state (Battery / Hybrid / Engine) Stateflow chart.

Enang and Bannister [4] survey modelling and control approaches for HEVs and highlight that a recurring weakness of purely threshold-based rule controllers is abrupt switching behaviour at the threshold boundary, which motivates the use of smoothing, hysteresis, or blending terms around mode transitions — consistent with the smoothing factor ($k = 0.2$) applied in this project's Hybrid and Engine states to avoid step discontinuities in the battery power command.

Bakht et al. [5] demonstrate the use of a Stateflow chart, MATLAB/Simulink's extended finite-state-machine tool, to implement the energy management strategy for a hybrid energy system, and report that the approach is straightforward to design and simulate while remaining well suited to representing event-driven switching between operating modes — supporting the choice of Stateflow as the implementation tool for the rule-based EMS logic in this project.

Taken together, the literature indicates that SOC-threshold-based, rule-driven control — implemented here as a three-state Stateflow chart with a smoothing term at mode transitions — is a recognized and well-precedented approach to EMS design, even though more advanced optimization-based and learning-based strategies can achieve better fuel economy at the cost of higher computational and design complexity [1]-[4].

4. Problem Statement

Electric and hybrid vehicles require a control strategy that decides, in real time, how much power should be drawn from the battery versus a secondary power source (engine), while:

- Keeping the battery State of Charge (SOC) within a safe operating band.
- Avoiding abrupt / step changes in power demand on either source.
- Responding to a time-varying load profile.

A purely battery-only or purely engine-only strategy cannot satisfy all three requirements simultaneously: relying on the battery alone risks deep discharge under sustained high load, while relying on the engine alone forfeits the efficiency benefit of using stored battery energy for moderate loads. The problem addressed in this project is therefore the design of a rule-based controller that switches between battery-only, blended, and engine-assisted operation based on SOC and load, without introducing discontinuous jumps in the commanded power at the switching instants.

5. Objectives

Based on the implemented model, the objectives of this project are:

- 1 To model the electrical subsystem of a battery–engine hybrid power source using Simscape Electrical.
- 2 To design a rule-based Energy Management Strategy using Stateflow with three operating modes — Battery, Hybrid, and Engine — switched based on SOC thresholds.
- 3 To implement smooth (non-step) power transitions between modes using a first-order smoothing factor.
- 4 To simulate the system response under a defined repeating load profile and observe the resulting battery power, engine power, and SOC behavior.

6. System Architecture

The model consists of four functional groups: a load profile source, a Stateflow-based EMS controller, an electrical power network (Simscape), and a SOC feedback/measurement path.

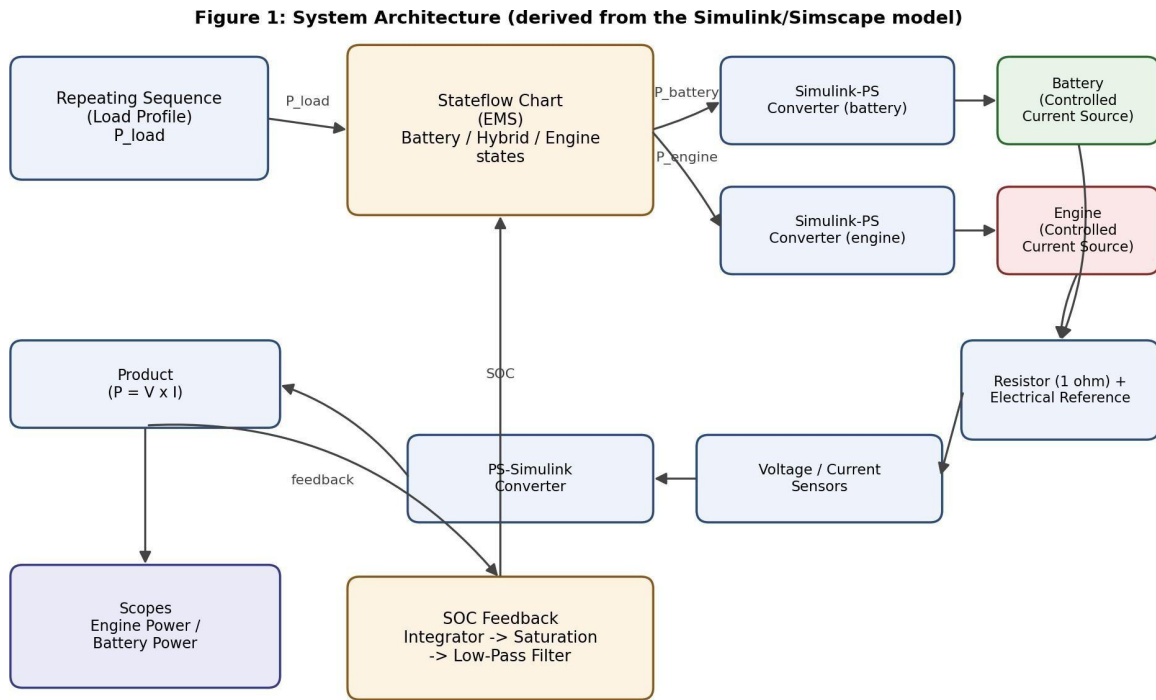


Figure 1: System architecture derived from the Simulink/Simscape model.

7. Components Used

The following blocks were identified directly from the model (system_root of HEMS_in_EV_Model.slx):

Block Name	Block Type / Library	Purpose in Model
Battery	Controlled Current Source (Simscape Electrical)	Represents the battery as an electrical power source
Engine	Controlled Current Source (Simscape Electrical)	Represents the engine as an electrical power source
Resistor	Electrical Resistor (Simscape Electrical)	Load/network element, Resistance = 1 ohm
Electrical Reference	Electrical Reference (Simscape Electrical)	Common ground reference for the electrical network
Current Sensor / Current Sensor1	Current Sensor (Simscape Electrical)	Measures current in the network
Voltage Sensor / Voltage Sensor1	Voltage Sensor (Simscape Electrical)	Measures voltage in the network
PS-Simulink Converter (x4)	Domain converter	Converts Simscape physical signals to Simulink signals
Simulink-PS Converter (x2)	Domain converter	Converts Simulink signals to Simscape physical signals
Solver Configuration	Simscape solver settings	Configures the physical network solver
Chart	Stateflow chart	Implements the EMS logic (Battery / Hybrid / Engine states)
Repeating Sequence	Repeating Table source	Generates the time-varying load profile
Integrator	Continuous Integrator	Computes/accumulates SOC, Initial condition = 80
Gain1	Gain	Gain = 1/3600 (0.00027778)
Gain	Gain	Gain = -1
Saturation	Saturation	Clamps SOC: Lower = 0, Upper = 100
Low-Pass Filter	eeGeneralControl block	Gain = 1, Time constant = 5 (smoothing)
Capacity	Constant	Constant value = 5 (units not specified in the model; see Section 11)
Product, Product2, Divide	Math Operations	Compute power ($P = V \times I$) and related ratios
Scope, Scope1, Scope2	Scope	Visualize Engine Power, Battery Power, and a third monitored signal (Scope2 not detailed in the supplied screenshots)

Simulink / Simscape Model — Top-Level View

Figure 0 shows the complete top-level Simulink/Simscape model (HEMS_in_EV_Model.slx). The left side contains the SOC estimation chain (Integrator → Saturation → Low-Pass Filter) and the Stateflow EMS chart; the right side implements the Simscape Electrical network with the Battery and Engine controlled current sources, sensors, and resistive load.

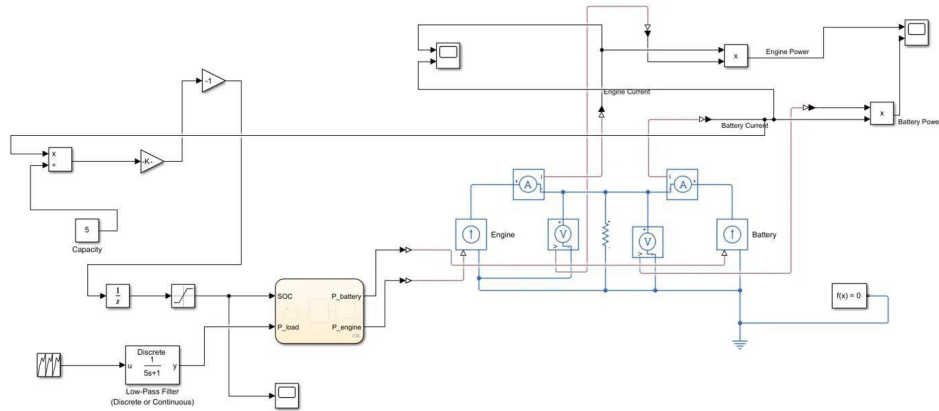


Figure 0: Top-level Simulink/Simscape model — HEMS_in_EV_Model.slx.

8. MATLAB / Simulink / Simscape Implementation

This section documents the parameters configured in each relevant block, exactly as set in the model.

8.1 Gain1

Element-wise gain block with Gain = $1/3600$ (approximately 0.00027778), used in the SOC computation path (likely converting a per-second quantity to a per-hour basis).

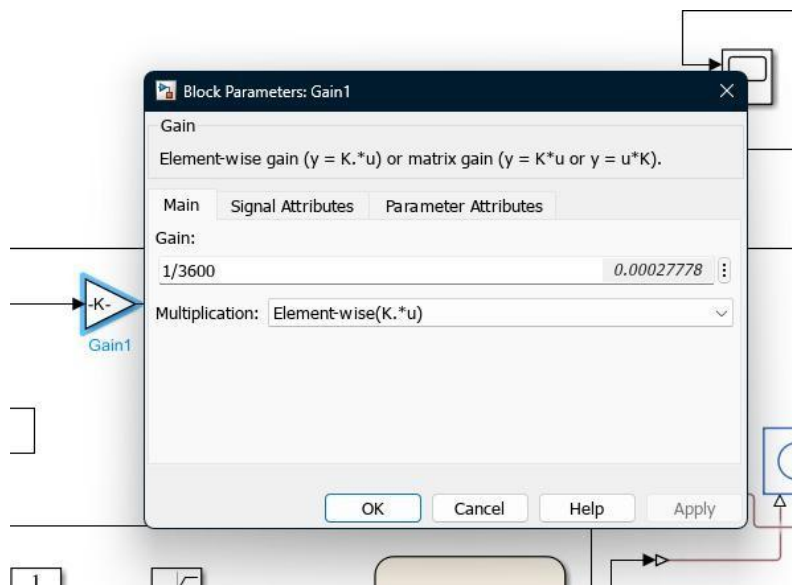


Figure 2: Gain1 block parameters.

8.2 Gain

Element-wise gain block with Gain = -1.

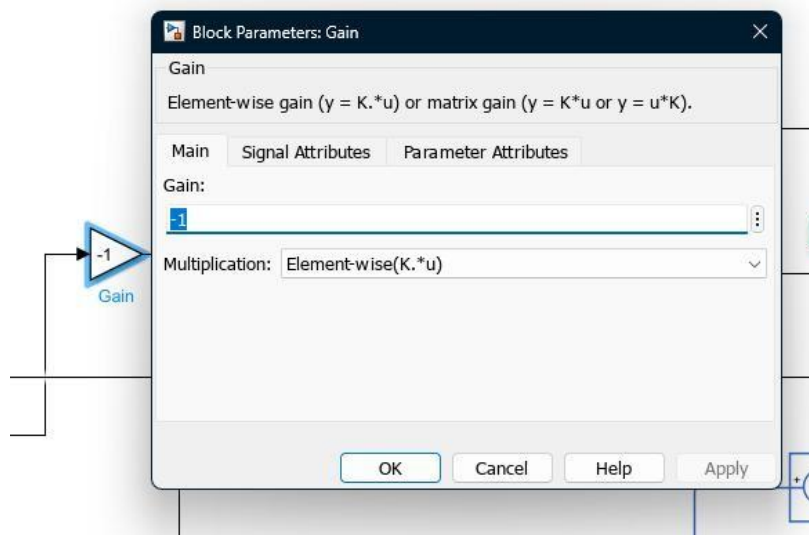


Figure 3: Gain block parameters.

8.3 Saturation

Clamps the SOC signal between Lower limit = 0 and Upper limit = 100, with "Treat as gain when linearizing" enabled.

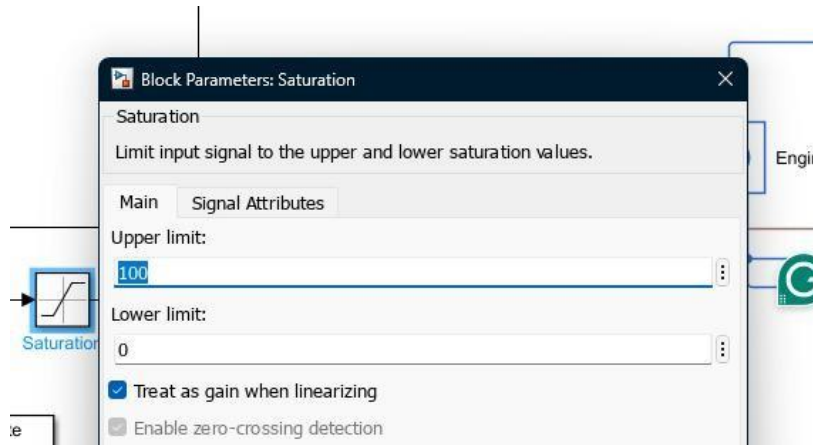


Figure 4: Saturation block parameters.

8.4 Low-Pass Filter (Discrete or Continuous)

- Gain = 1
- Time constant = 5
- Upper saturation limit = inf, Lower saturation limit = -inf
- Initialization = Inherited from block input
- Sample time = -1 (inherited)

Block Parameters: Low-Pass Filter (Discrete or Continuous)

Low-Pass Filter (Discrete or Continuous) (mask) (link)

This block models a low-pass filter.

The saturation limit is anti-windup. For no saturation limits, set Upper saturation limit to inf, and Lower saturation limit to -inf. To use windup saturation limits, configure this block to have no saturation limits, and connect a Saturation block to the output. For inherited discrete-time, set Sample time to -1. For discrete-time, set Sample time to a positive value. For continuous-time, set Sample time to zero.

If the time constant is set to zero, then the block passes the input directly to the output with no dynamics.

For more information on using this block to represent an IEEE Std 421.5 simple time constant, click the Help button.

Parameters

Gain

Time constant

5

Upper saturation limit

inf

Lower saturation limit

-inf

Initialization

Inherited from block input

Sample time (-1 for inherited)

-1

OK

Cancel

Help

Apply

Figure 5: Low-pass filter block parameters.

8.5 Repeating Sequence (Load Profile)

Defines the load demand profile used in the simulation:

Time (s)	0	5	10	15	20
Output value	1000	1500	2000	1800	2200

This sequence repeats periodically for the duration of the simulation.

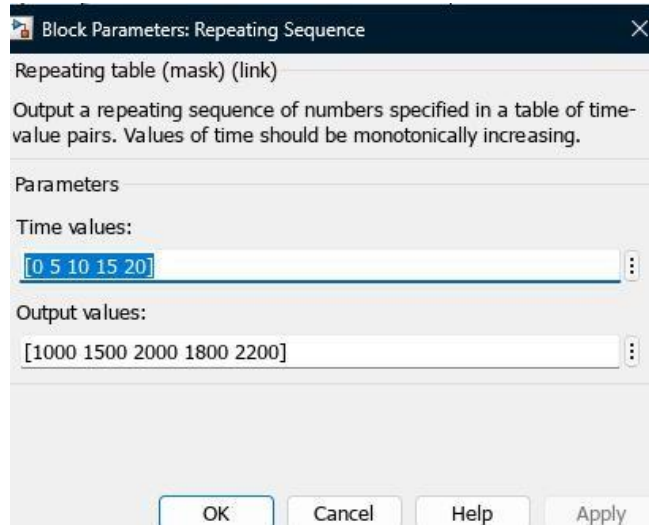


Figure 6: Repeating Sequence block parameters.

8.6 Integrator (SOC State)

- External reset: none
- Initial condition source: internal
- Initial condition = 80 (i.e., simulation starts at SOC = 80%)
- Absolute tolerance: auto

Block Parameters: Integrator

Integrator

Continuous-time integration of the input signal.

Parameters

External reset: none

Initial condition source: internal

Initial condition: 80

☐ Limit output

☐ Wrap state

☐ Show saturation port

☐ Show state port

Absolute tolerance: auto

☐ Ignore limit and reset when linearizing

☒ Enable zero-crossing detection

State Name: (e.g., 'position')

"

OK Cancel Help Apply

Figure 7: Integrator block parameters.

8.7 Capacity (Constant)

- Constant value = 5
- Interpret vector parameters as 1-D: enabled
- Sample time: inf

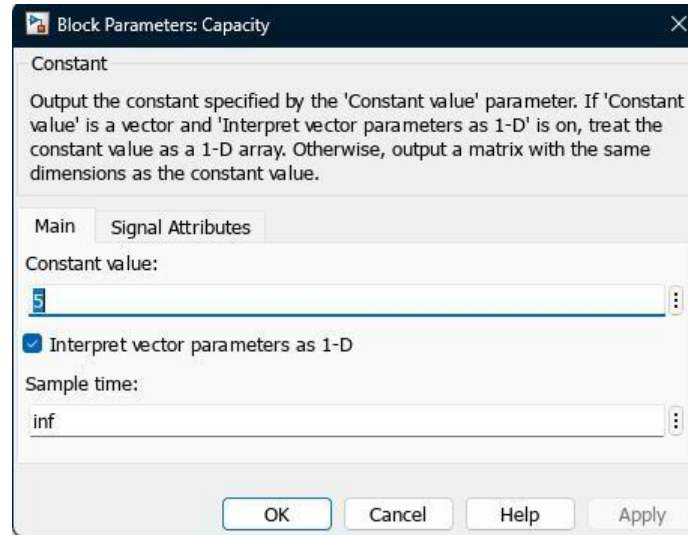


Figure 8: Capacity constant block parameters.

8.8 Resistor (Simscape Electrical)

Resistance = 1 ohm.

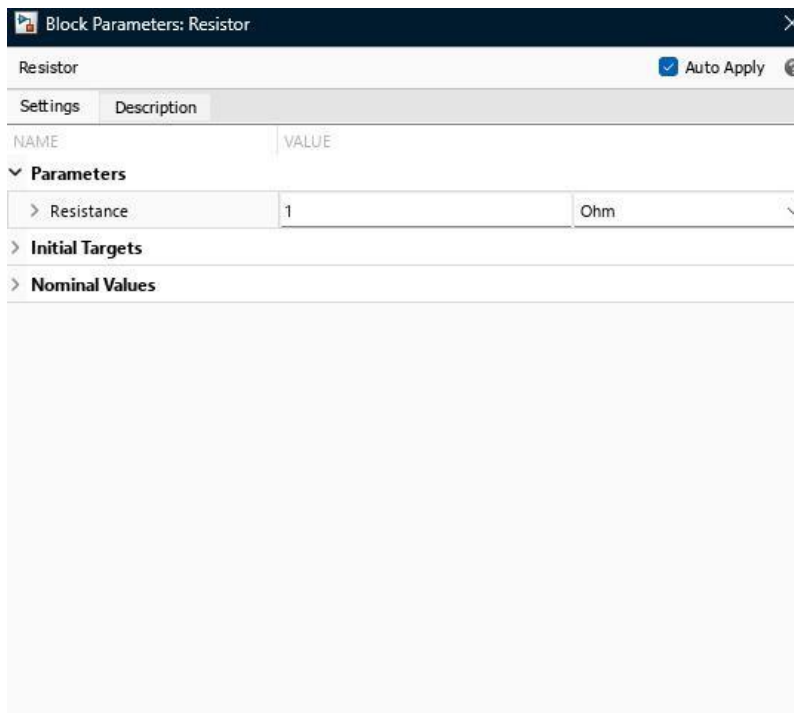


Figure 9: Resistor block parameters.

8.9 Solver Configuration

- Equation formulation: Time
- Index reduction method: Derivative replacement
- Consistency tolerance: Model AbsTol and RelTol
- Tolerance factor: 0.001
- Linear Algebra: auto
- Delay memory budget: 1024 kB
- Apply filtering at 1-D/3-D connections when needed: enabled (Filtering time constant = 0.001)

Solver Configuration	
Settings	Description
NAME	VALUE
Equation formulation	Time
Index reduction method	Derivative replacement
☐ Start simulation from steady state	
Consistency tolerance	Model AbsTol and RelTol
Tolerance factor	0.001
☐ Use local solver	
☐ Use fixed-cost runtime consistency iterations	
Linear Algebra	auto
Delay memory budget [kB]	1024
☒ Apply filtering at 1-D/3-D connections when needed	
Filtering time constant	0.001
> Multibody	

Figure 10: Solver Configuration block parameters.

9. Energy Management Strategy (EMS)

The EMS is implemented as a Stateflow chart with three states: Battery, Hybrid, and Engine. The chart starts in the Battery state (indicated by the default transition arrow).

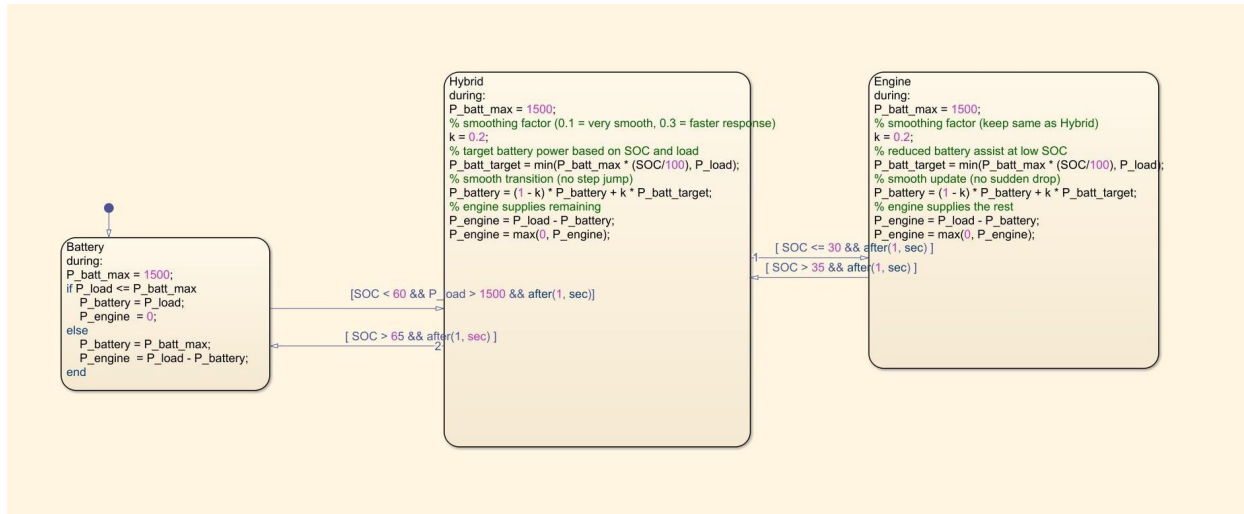


Figure 11: Stateflow chart implementing the EMS logic.

9.1 State: Battery (initial state)

```
during:
P_batt_max = 1500;
if P_load <= P_batt_max
    P_battery = P_load;
    P_engine = 0;
else
    P_battery = P_batt_max;
    P_engine = P_load - P_battery;
end
```

Transition out: [SOC < 60 && P_load > 1500 && after(1, sec)] → Hybrid

9.2 State: Hybrid

```
during:
P_batt_max = 1500;
k = 0.2; % smoothing factor (0.1 = very smooth, 0.3 = faster
P_batt_target = min(P_batt_max * (SOC/100), P_load); % target battery power based on SOC and load
P_battery = (1 - k) * P_battery + k * P_batt_target; % smooth transition (no step jump)
P_engine = P_load - P_battery; % engine supplies remaining
P_engine = max(0, P_engine);
```

Transitions out:

- [SOC <= 30 && after(1, sec)] → Engine
- [SOC > 65 && after(1, sec)] → Battery

9.3 State: Engine

```
during:
P_batt_max = 1500;
k = 0.2;                                     % smoothing factor (kept same as Hybrid)
P_batt_target = min(P_batt_max * (SOC/100), P_load); % reduced battery assist at low SOC
P_battery = (1 - k) * P_battery + k * P_batt_target; % smooth update (no sudden drop)
P_engine = P_load - P_battery;               % engine supplies the rest
P_engine = max(0, P_engine);
```

Transition out: [SOC > 35 && after(1, sec)] → Hybrid

9.4 Summary of Transition Logic

From	To	Condition
Battery	Hybrid	SOC < 60 && P_load > 1500 && after(1, sec)
Hybrid	Battery	SOC > 65 && after(1, sec)
Hybrid	Engine	SOC <= 30 && after(1, sec)
Engine	Hybrid	SOC > 35 && after(1, sec)

A common smoothing factor $k = 0.2$ is used in both the Hybrid and Engine states to filter the battery power target (P_{batt_target}) into the actual battery power command ($P_{battery}$), avoiding step discontinuities when switching modes.

10. Working Principle

- 1 The Repeating Sequence block generates a load demand signal P_{load} that cycles through the defined time-value pairs.
- 2 The Stateflow chart receives P_{load} and the current SOC, and — depending on which of the three states is active — computes $P_{battery}$ and P_{engine} .
- 3 In the Battery state, the battery alone supplies the load as long as it stays within P_{batt_max} (1500); any excess is passed to the engine.
- 4 As SOC drops and/or load exceeds 1500, the system transitions into the Hybrid state, where the battery power command is smoothly blended toward a SOC-proportional target ($P_{batt_max} * SOC/100$) using the smoothing factor $k = 0.2$, with the engine covering the remainder.
- 5 If SOC falls to 30% or below, the system transitions into the Engine state, reducing battery assistance further while still applying the same smoothing logic.
- 6 The commanded $P_{battery}$ and P_{engine} values are converted from Simulink signals to physical (Simscape) signals via Simulink-PS Converter blocks and drive the Battery and Engine controlled current sources.
- 7 Voltage Sensor and Current Sensor blocks measure the resulting electrical quantities, which are converted back to Simulink signals (PS-Simulink Converter) and multiplied (Product blocks) to compute instantaneous power.
- 8 SOC is derived through an Integrator (initial value 80) fed by a scaled version of the battery power (via $Gain1 = 1/3600$ and $Gain = -1$), passed through a Saturation block (0-100) to keep SOC physically valid, and a Low-Pass Filter (time constant = 5) for smoothing — this SOC value feeds back into the Stateflow chart, closing the control loop.

11. Simulation Setup

Parameter	Value
Initial SOC	80%
Battery capacity parameter (Capacity constant)	5
Battery max power (P_batt_max, used in Stateflow logic)	1500
Smoothing factor (k)	0.2
Network resistance	1 ohm
Load profile (time, s)	0, 5, 10, 15, 20
Load profile (value)	1000, 1500, 2000, 1800, 2200
SOC saturation limits	0 (lower) - 100 (upper)
SOC low-pass filter time constant	5
Total simulation time	3000 (as read from Scope1)
Solver configuration	Equation formulation: Time; Tolerance factor: 0.001; Linear Algebra: auto

The unit of the Capacity constant (value = 5) and the time base of the 3000-unit simulation horizon were not explicit in the supplied block-parameter screenshots; given the magnitude and the use of a 1/3600 gain elsewhere in the SOC path (consistent with a seconds-to-hours conversion), it is most consistent with the rest of the model for the capacity to be in ampere-hours (Ah) and the simulation horizon to be in seconds, but this should be verified against the model's actual configuration before being stated as fact in a viva. Similarly, the solver type and fixed/variable step size are set in the Simulink Model Configuration Parameters dialog rather than in the Simscape Solver Configuration block documented in Section 8.9, and were not part of the supplied screenshots; this can be read off directly from the model under Modeling > Model Settings > Solver.

12. Results and Discussion

The simulation output is captured in Scope1, which displays Engine Power (top plot, red) and Battery Power (bottom plot, green) over a simulation horizon of 0-3000 (time base as configured), sampled at the simulation's sample rate ("Sample based, T = 3000.0000").

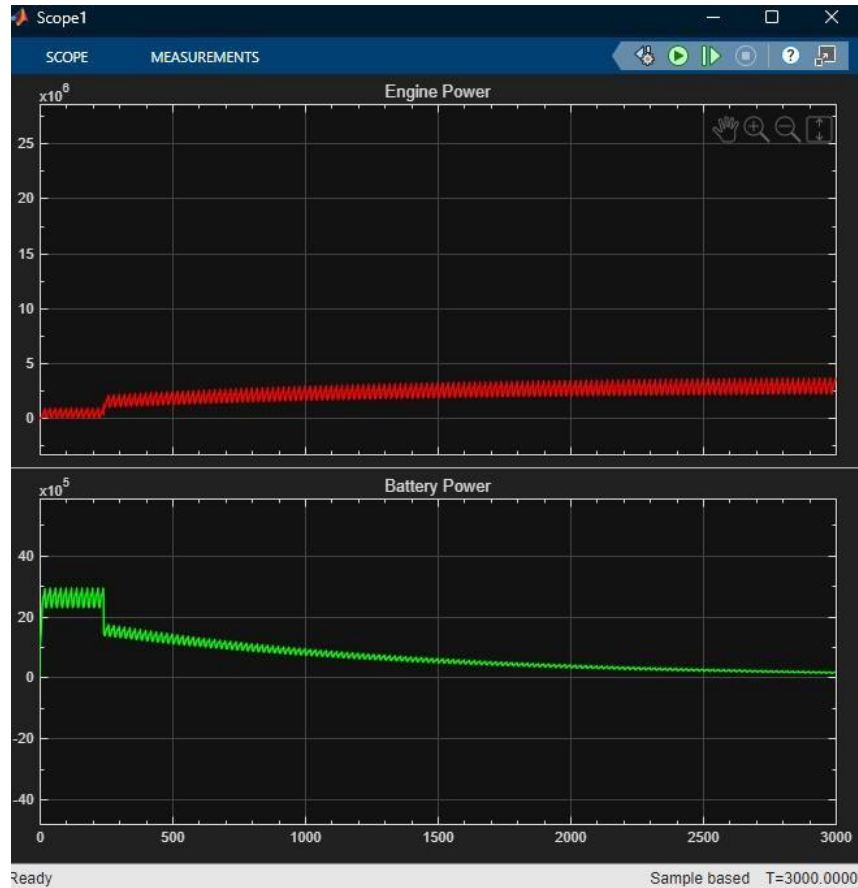


Figure 12: Engine Power and Battery Power vs. time (Engine_Battery_Power_Output).

Observed Trends

- Engine Power (top plot, y-axis scaled $\times 10^6$): starts near zero, rises during the initial part of the simulation, and then stabilizes into a repeating ripple pattern around a near-constant level for the remainder of the run. The ripple corresponds to the periodic nature of the Repeating Sequence load profile (period = 20 s).
- Battery Power (bottom plot, y-axis scaled $\times 10^5$): starts at its highest level, oscillating with the same ripple pattern, and progressively decreases over time, approaching a low/near-zero level by the end of the simulation window.
- The overall trend — battery power decreasing while engine power increases and then plateaus — is consistent with the EMS logic: as SOC depletes from its initial value of 80%, the controller transitions from the Battery state, through Hybrid, and increasingly relies on the Engine state to meet the load.
- The repeating sawtooth-like ripple visible in both traces is a direct consequence of the Repeating Sequence load profile cycling through its 5-point table every 20 s.

SOC Trajectory — SOC_Scope Output

Figure 13 shows the battery State of Charge (SOC) over the full 3000-second simulation horizon, as captured in Scope2. Starting at 80 %, the SOC decays continuously as the battery supplies load power. The smooth, monotonically decreasing profile is consistent with the Low-Pass Filter (time constant = 5) applied in the SOC feedback path and confirms that the saturation block successfully prevents the value from going negative. The rate of decay visibly slows as the EMS transitions through the Hybrid and Engine states, reducing reliance on the battery at lower SOC levels.

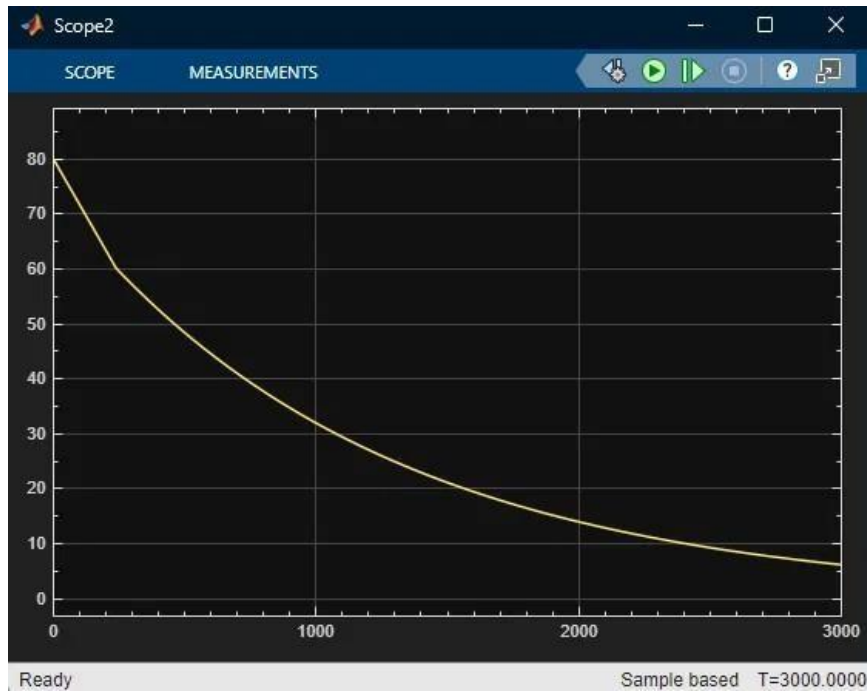


Figure 13: Battery SOC vs. time (0–3000 s), as displayed in Scope2. Initial SOC = 80 %; the curve decays smoothly toward ~5 % by the end of the simulation.

- The battery power trace shows a distinct change in slope at roughly the 200-300 s mark, where the oscillation envelope drops from its initial high band to a lower band before beginning its slow decline. Given the $SOC < 60$ transition condition out of the Battery state, this visible kink is consistent with the model leaving the initial Battery state and entering the Hybrid state relatively early in the run, after which the slower, continuous decline reflects the Hybrid state's SOC-proportional blending of battery power as SOC continues to fall.
- Neither trace shows a second, sharper transition later in the run (which would be expected when $SOC \leq 30$ triggers the move into the pure Engine state); by the end of the simulation window the battery power has decayed close to zero, which is consistent with either a late-stage Engine-state condition or simply a very low Hybrid-state SOC-proportional target — this cannot be distinguished from the power plots alone, since SOC itself is not one of the plotted signals in Scope1.

Note: The magnitude of the plotted power values (engine power in the 10^6 range, battery power in the 10^5 range) is large relative to the Repeating Sequence load values (1000-2200). This is expected behaviour for the Simscape network as configured: with controlled current sources driving a 1 ohm resistor, the resulting voltage and current magnitudes (and hence $V \times I$ power) scale up quickly for current values in the hundreds-to-thousands range, so the 10^5 - 10^6 scope readings are plausible outputs of this particular network rather than necessarily an error — though it is still worth confirming that the controlled-current-source inputs are in the intended units (e.g., amperes vs. some other scaled quantity) given how large the resulting power figures are.

13. Advantages

- The smoothing factor ($k = 0.2$) applied in the Hybrid and Engine states prevents step-jumps in battery power command when the EMS switches modes, as documented directly in the Stateflow code ("smooth transition (no step jump)") — this directly addresses the abrupt-switching weakness of simple threshold-only rule controllers noted in the literature [4].
- The three-state (Battery / Hybrid / Engine) structure keeps the control logic modular and easy to trace/debug compared to a single monolithic control law, and maps cleanly onto the if-then/threshold style of rule-based EMS design described in [1] and [3].
- The architecture separates the control logic (Stateflow, Simulink) from the physical electrical network (Simscape), allowing the EMS strategy to be modified independently of the electrical model.
- Because the strategy is rule-based rather than optimization-based, it requires no a-priori knowledge of the future load/drive profile and is computationally light enough to run in real time on simple embedded hardware, consistent with the rule-based vs. optimization-based trade-off summarized in [1] and [2].
- Using Stateflow rather than hand-written conditional logic in a MATLAB Function block makes the mode structure and transition conditions visually explicit, which simplifies verification and viva-style walkthroughs of the control logic.

14. Limitations

- The EMS uses fixed, hand-tuned SOC thresholds (30, 35, 60, 65) and a fixed smoothing factor ($k = 0.2$); it is a rule-based strategy rather than an optimization-based one, so it does not guarantee minimum fuel/energy consumption — a known limitation of rule-based EMS design relative to optimization-based methods [2], [3].
- The electrical network is simplified to a single resistor load with controlled current sources for the battery and engine — it does not model battery internal resistance, thermal effects, or detailed engine dynamics (fuel consumption, emissions, transient response).
- The unit/scale of the Capacity constant (value = 5) is not explicit in the model and should be clarified before the project's numerical results are quoted in a report or viva.
- SOC is not one of the signals plotted on the available scopes, so the exact instants of mode transition can only be inferred indirectly from the shape of the power traces rather than read directly off a SOC-vs-time plot.
- The load profile used for testing is a short, hand-defined 5-point repeating sequence rather than a standard drive cycle (e.g., NEDC, WLTP, or a real-world logged profile), which limits how representative the simulation results are of real driving conditions.

15. Future Scope

- Replace or augment the fixed SOC thresholds with an optimization-based layer (e.g., dynamic programming, an Equivalent Consumption Minimization Strategy, or model predictive control) to approach minimum-fuel operation while retaining the existing rule-based structure as a real-time-feasible fallback, in line with the hybrid rule/optimization approaches discussed in the literature [2].
- Validate the controller against standard drive cycles (e.g., NEDC, WLTP, or a representative logged urban/highway profile) instead of the short hand-defined repeating sequence used here, to assess performance under more realistic load conditions.
- Extend the electrical network with a more detailed battery model (internal resistance, open-circuit-voltage-vs-SOC curve, thermal effects) and a representative engine/generator model, rather than the simplified controlled-current-source representation used in this project.
- Add SOC and mode-state signals to the scopes (or a dedicated logging setup) so that transition instants and SOC trajectory can be verified directly rather than inferred from the power traces.
- Tune or adapt the smoothing factor k and the SOC thresholds (currently fixed at 0.2 and 30/35/60/65 respectively) using simulation-based sensitivity analysis, rather than the hand-picked values used in the present model.
- Investigate intelligent/learning-based EMS variants (fuzzy logic, reinforcement learning) as a longer-term extension, following the trend toward intelligent control strategies noted in [2].

16. Applications

- Power-split control for hybrid electric vehicles (HEVs) and plug-in hybrid electric vehicles (PHEVs), which is the direct application context of the model developed in this project.
- Range-extended electric vehicles (REEVs), where a small onboard engine/generator supplements the battery once SOC falls below a defined threshold — structurally similar to the Engine-state behaviour implemented here.
- Energy management for other battery + secondary-source hybrid systems with the same SOC/load decision structure, such as fuel-cell/battery hybrid powertrains or battery/diesel-generator hybrid microgrids, where rule-based, Stateflow-style controllers have also been used in the literature [5].
- As a teaching/demonstration model for rule-based EMS concepts, given its compact three-state Stateflow structure and clear separation between control logic and the Simscape electrical network.

17. Conclusion

A rule-based Hybrid Energy Management System was modeled in Simulink using a three-state Stateflow chart (Battery, Hybrid, Engine) to coordinate power-sharing between a battery and an engine source, based on SOC and load demand. The electrical network was implemented using Simscape Electrical components, and SOC feedback was derived through an integrator, saturation, and low-pass filter chain. Simulation results (Scope1) show the expected qualitative behavior — battery power dominant at high SOC, with a gradual shift toward engine power as SOC depletes — confirming that the implemented

mode-switching logic functions as designed.

The project demonstrates that a SOC-threshold-based rule strategy, of the type widely described in HEV energy-management literature [1]-[4], can be implemented compactly and transparently using Stateflow, and that the smoothing factor included in the Hybrid and Engine states successfully avoids the abrupt power-command discontinuities that simple threshold-only controllers are prone to. The main outstanding items for a complete evaluation are: confirming the unit of the Capacity parameter and the solver settings used (Section 11), and instrumenting SOC directly on a scope so that the inferred mode-transition instants discussed in Section 12 can be verified numerically rather than read qualitatively off the power traces.

18. References

- [1] S. G. Wirasingha and A. Emadi, "Classification and Review of Control Strategies for Plug-In Hybrid Electric Vehicles," *IEEE Transactions on Vehicular Technology*, vol. 60, no. 1, pp. 111-122, Jan. 2011.
- [2] Q. Xue, X. Zhang, T. Teng, J. Zhang, Z. Feng, and Q. Lv, "A Comprehensive Review on Classification, Energy Management Strategy, and Control Algorithm for Hybrid Electric Vehicles," *Energies*, vol. 13, no. 20, art. 5355, Oct. 2020.
- [3] M. Sabri, K. A. Danapalasingam, and M. F. Rahmat, "A review on hybrid electric vehicles architecture and energy management strategies," *Renewable and Sustainable Energy Reviews*, vol. 53, pp. 1433-1442, 2016.
- [4] W. Enang and C. Bannister, "Modelling and control of hybrid electric vehicles (A comprehensive review)," *Renewable and Sustainable Energy Reviews*, vol. 74, pp. 1210-1239, 2017.
- [5] M. P. Bakht, Z. Salam, A. R. Bhatti, W. Anjum, S. A. Khalid, and N. Khan, "Stateflow-Based Energy Management Strategy for Hybrid Energy System to Mitigate Load Shedding," *Applied Sciences*, vol. 11, no. 10, art. 4601, May 2021.
- [6] MATLAB, Simulink, Simscape, and Stateflow documentation, The MathWorks, Inc. [Online]. Available: <https://www.mathworks.com>